

SERVER-SIDE RENDERING Z WYKORZYSTANIEM NODE.JS

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	2
Ponowne klonowanie repozytorium.....	2
Pobranie i uruchomienie PNPM.....	2
Pierwsze kroki z Express.js.....	2
Routing i szablony.....	4
Portowanie aplikacji z LAB G.....	5
Implementacja.....	7
Commit projektu do GIT.....	11
Podsumowanie.....	11

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- wykorzystanie menedżera pakietów pnpm;
- instalacja i uruchomienie Express.js;
- zarządzanie routingami w Express.js;
- wykorzystanie szablonów EJS;
- portowanie aplikacji webowych CRUD pomiędzy PHP a JS.

W praktycznym wymiarze uczestnicy wykorzystają prosty framework Express.js do odtworzenia funkcjonalności naiwnego frameworka z poprzedniego laboratorium.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie zasad instalacji i uruchamiania programów w node. Powtórzenie zasad działania npm i pnpm. Różnice pomiędzy npm i pnpm.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A -> F9**.

PONOWNE KLONOWANIE REPOZYTORIUM

W związku z problemami wydajnościowymi w odczycie plików na macierzy dyskowej **I:**, to laboratorium również musimy wykonać na pulpicie. Jest to **bardzo zła** praktyka, ale niestety administratorom nie udało się znaleźć innego rozwiązania ani permanentnego obejścia. Wyłącznie jeśli korzystasz z pulpitu zdalnego do realizacji zadań, upewnij się, że Twoje repozytorium jest sklonowane na pulpit, tj. do katalogu analogicznego do **I:\ptw**.

Na tym etapie warto w ustawieniach tej karty pracy zmienić wartość niestandardowej właściwości **labmdir** oraz **labdir**, po czym odświeżyć wszystkie pola w dokumencie **Ctrl+A, F9**.

POBRANIE I URUCHOMIENIE PNPM

W tym zadaniu przećwiczymy korzystanie z innego menedżera pakietów dla node – **pnpm**. Otwórz terminal i przejdź do katalogu repozytorium, tj. **I:\ptw**. Zainstaluj pnpm poleceniem:

```
npm install -g pnpm@latest-10
```

Następnie sprawdź poprawność instalacji poleceniem:

```
pnpm -v
```

Powinna wyświetlić się wersja programu, przykładowo:

```
PS C:\Users\akarczmarczyk\Desktop\ptw> pnpm -v
10.33.4
```

PIERWSZE KROKI Z EXPRESS.JS

Utwórz katalog **I:\ptw\lab-i\hello**. Przejdź do niego terminalem i zainstaluj Express.js w wersji 5:

```
pnpm install express@5
```

Spodziewany efekt:

```
PS C:\Users\akarczmarczyk\Desktop\ptw\lab-i\hello> pnpm install express@5
Packages: +65
Progress: resolved 65, reused 0, downloaded 65, added 65, done
dependencies:
+ express 5.2.1
Done in 2.3s using pnpm v10.33.4
```

Następnie doinstaluj zależność prettier:

```
pnpm install prettier@3
```

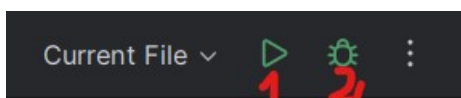
Następnie otwórz katalog `I:\ptw\lab-i\hello` w PhpStorm. Utwórz plik `hello.js` o następującej treści (koniecznie wykorzystaj swój numer albumu jako numer portu):

```
import express from 'express'
const app = express()
const port = 57722
app.get('/', (req, res) => {
  res.send('Hello World from Express!')
})
app.listen(port, () => {
  console.log(`Hello World app listening on port ${port}`)
})
```

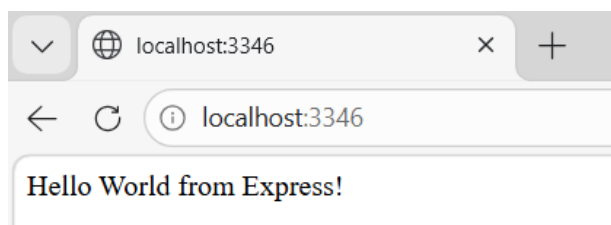
Teraz skonfigurujemy Prettier. Wejdź do ustawień PhpStorm i w Languages & Frameworks > JavaScript > Prettier wybierz konfigurację automatyczną. Zapisz zmiany. Kliknij w kod `hello.js` i wykorzystaj skrót klawiszowy `Ctrl + Alt + Shift + P`. Kod powinien ulec automatycznemu reformatowaniu.

Cofnij i ponów operację kilka razy. Jakie zmiany zachodzą po wykorzystaniu Prettiera?

Teraz uruchom serwer klikając na przycisk (1) poniżej:

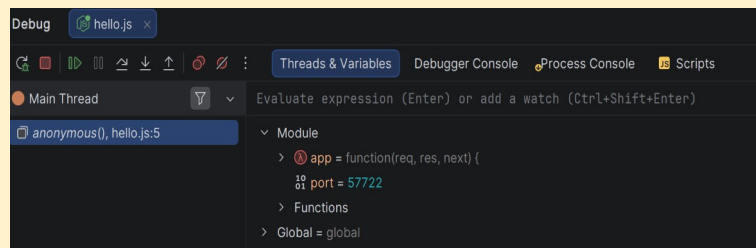


Przeczytaj w panelu logów, czy uruchomienie serwera powiodło się, a następnie otwórz w przeglądarce stronę `http://localhost:57722`. Oczekiwany efekt:



Przetestujemy teraz debugowanie. Ustaw pułapkę w handlerze route'a `/` (linia 5). Zatrzymaj serwer i uruchom ponownie przyciskiem (2) powyżej. Ponownie wejdź na stronę `http://localhost:57722`. Aplikacja powinna zatrzymać się, a panel `Threads & Variables` wypełniony będzie dostępnymi zmiennymi i ich wartościami.

Wstaw zrzut ekranu kodu aplikacji w PhpStorm, z uruchomionym debuggerem, zatrzymanym wewnątrz handlera ścieżki `„/”`. Upewnij się, że kod jest sformatowany Prettierem, debugger faktycznie jest zatrzymany we wskazanej linii, a w panelu `Threads & Variables` dostępne są bieżące wartości zmiennych.



Punkty:

0

1

ROUTINGI I SZABLONY

Obecnie aplikacja obsługuje tylko jedną ścieżkę (/), a zwracana odpowiedź jest zakodowana w handlerze na sztywno. W kolejnym kroku dodamy nowy routing oraz wykorzystamy szablony EJS.

Zainstaluj w projekcie EJS:

```
pnpm install ejs@5
```

Zmodyfikuj kod aplikacji zgodnie z poniższym:

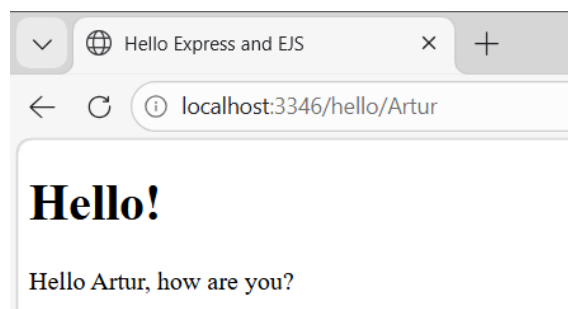
<pre>import express from "express"; const app = express(); const port = 3346; app.get("/", (req, res) => { res.send("Hello World from Express!"); }); app.listen(port, () => { console.log(`Hello World app listening`); });</pre>	<pre>1 1 2 2 3 3 >> 4 4 5 5 6 6 >> 7 7 8 8 9 9 10 10 11 12 13 14</pre>	<pre>import express from "express"; const app = express(); const port = 3346; app.set("view engine", "ejs"); app.get("/", (req, res) => { res.send("Hello World from Express!"); }); app.get("/hello/:name", (req, res) => { const name = req.params.name; res.render("hello", { name }); }); app.listen(port, () => { console.log(`Hello World app listening o`); });</pre>
--	--	---

Następnie utwórz plik `views/hello.ejs` o następującej treści:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Hello Express and EJS</title>
</head>
<body>
<h1>Hello!</h1>
<p>Hello <%= name %>, how are you?</p>
</body>
```

</html>

Zrestartuj serwer i wejdź na stronę <http://localhost:57722/hello/Jakub>. Spodziewany efekt:



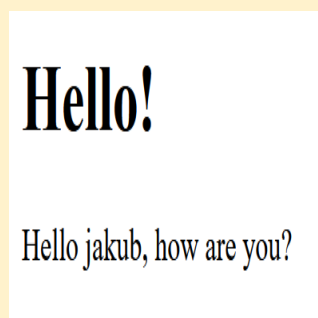
Wstaw zrzut ekranu kodu hello.js na tym etapie:

```
import express from 'express'
const app = express()
const port : number = 57722
app.set("view engine", "ejs");
app.get('/', (req, res) : void => {
  res.send('Hello World from Express!')
})
app.get("/hello/:name", (req, res) : void => {
  const name = req.params.name;
  res.render("hello", { name });
});
app.listen(port, () : void => {
  console.log(`Hello World app listening on port ${port}`);
});
```

Wstaw zrzut ekranu kodu views/hello.ejs:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, height=device-height">
  <title>Hello Express and EJS</title>
</head>
<body>
<h1>Hello!</h1>
<p>Hello <%= name %>, how are you?</p>
</body>
</html>
```

Wstaw zrzut ekranu strony <http://localhost:57722/hello/Jakub> w przeglądarce:



W 3-5 zdaniach omów która część kodu odpowiada za to, że routing /hello/Jakub zaczął działać, a imię zaczęło się wyświetlać w odpowiedzi:

...3-5 zdań...

Punkty:	0	1
---------	---	---

PORTOWANIE APLIKACJI Z LAB G

W ostatniej części laboratorium przygotujemy kompletną aplikację CRUD (Create, Read, Update, Delete) na podstawie LAB G. Do rozpoczęcia nowego projektu wykorzystamy generator aplikacji Express. Przejdź terminalem do katalogu `I:\ptw\lab-i` i wykonaj polecenia:

```
npx express-generator@4 --view=ejs --css=less crud
cd crud
pnpm install
```

Otwórz w PhpStorm projekt z katalogu `I:\ptw\lab-i\crud`. Zapoznaj się ze strukturą plików i katalogów nowoutworzonego projektu.

W pliku `bin/www` zmodyfikuj port 3000 na swój własny (**57722**). Otwórz plik `bin/www` w edytorze i uruchom serwer w trybie zwykłym lub debug wybranym przyciskiem (1) lub (2) powyżej.

Odpowiedz na pytania:

Jaka wersja Express.js została wykorzystana? 5

Czy to najświeższa wersja? Nie wiem

Jak zmienić domyślny port aplikacji z 3000 na **57722**? w pliku `bin/www`

Gdzie znajdują się style less?

`public/stylesheets/`

Jak pogrupowane są routingi i ich handlers? Jako pliki w `routes` oraz `views`

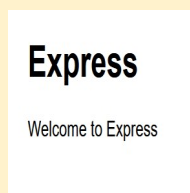
Gdzie znajdują się widoki EJS?

`views`

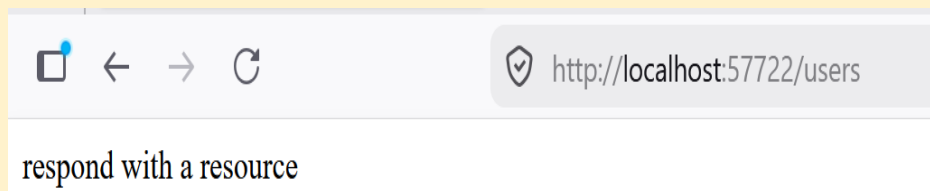
Dlaczego plik `bin/www` nie ma rozszerzenia `.js`?

Nie wiem? ciekawe

Wstaw zrzut ekranu strony `http://localhost:57722/` w przeglądarce:



Wstaw zrzut ekranu strony `http://localhost:57722/users` w przeglądarce:



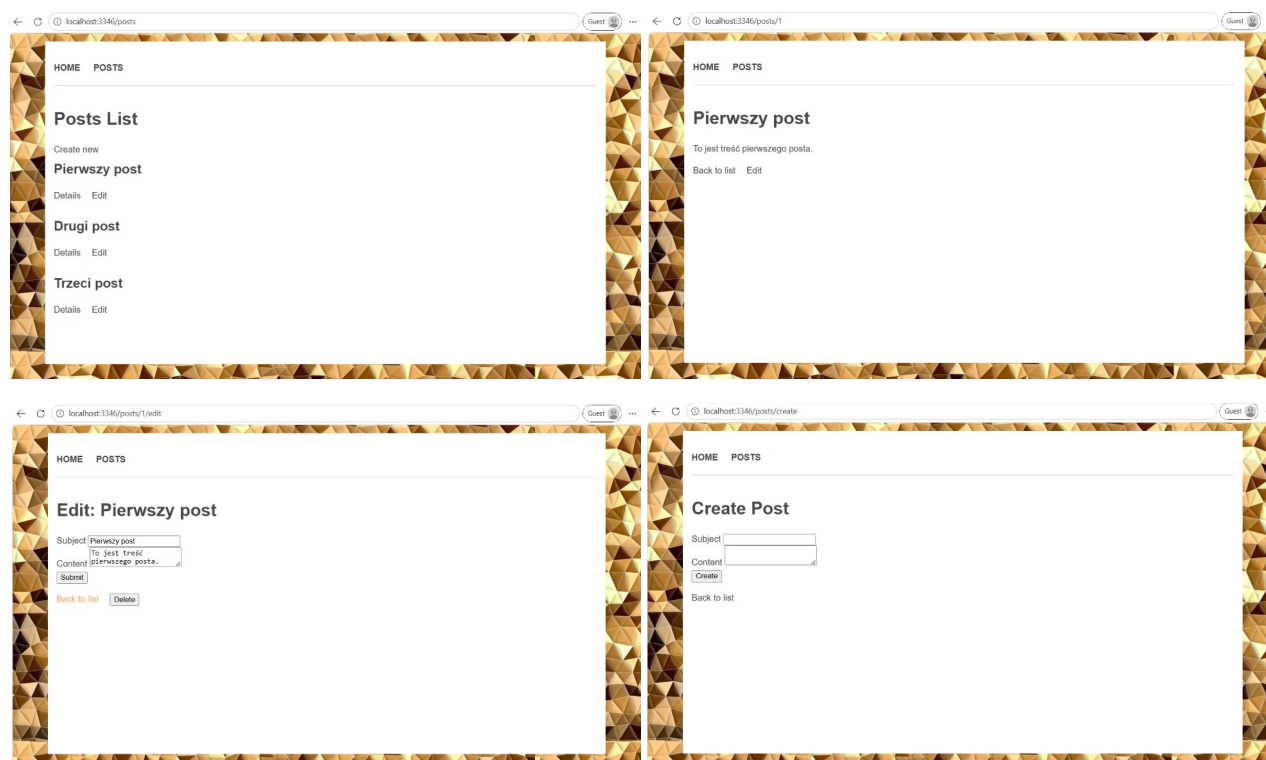
Upewnij się, że na zrzutach widać pasek adresu z Twoim numerem indeksu.

Punkty:	0	1
---------	---	---

IMPLEMENTACJA

W pozostałej części laboratorium, przenieś layout, style, migracje SQL (baza SQLite) oraz wszystkie operacje CRUD dotyczące Twojego modelu z aplikacji PHP do aplikacji w Express.js (tylko Twój model, bez postów).

Przykładowy efekt:



Hint! Obsługę SQLite w JS można osiągnąć na wiele różnych sposobów. Ja wybrałem taki:

```
posts.js
1  var express : e | () => core.Express = require('express');
2  var router : Router = express.Router();
3  const { DatabaseSync } = require('node:sqlite');
4  const path = require('node:path');
5  const dbPath : string = path.resolve(__dirname, '..', 'data.db');
6  const db : DatabaseSync = new DatabaseSync(dbPath);

18 router.post( path: '/create', handlers: function(req : Request<P, ResBody, ReqBody, Re
19   try {
20     const { subject, content } = req.body;
21
22     const result : StatementResultingChanges = db
23       .prepare( sql: 'INSERT INTO post (subject, content) VALUES (?, ?)'
24       .run(subject, content);
25
26     res.redirect( url: '/posts/' + result.lastInsertRowid);
27   } catch (err) {
28     next(err);
29   }
30 });
```

Wstaw zrzut ekranu kodu routingu i handlera listy:

```
var express : e | () => core.Express = require('express');
var router : Router = express.Router();
var Car : Car | {...} = require('../models/car');

/* GET cars listing. */
router.get( path: '/', handlers: function(req : Request<P, ResBody, ReqBody, ReqQuery>): void {
  const cars : Car[] = Car.findAll();
  res.render( view: 'cars/index', options: { title: 'Cars List', cars: cars });
});

/* GET create car form. */
router.get( path: '/create', handlers: function(req : Request<P, ResBody, ReqBody, ReqQuery>): void {
  res.render( view: 'cars/create', options: { title: 'Create New Car', cars: cars });
});

/* POST create car. */
router.post( path: '/create', handlers: function(req : Request<P, ResBody, ReqBody, ReqQuery>): void {
  const { subject, content, carModel } = req.body;
  const car : Car = new Car({ subject, content, carModel });
  car.save();
  res.redirect( url: '/cars');
});
```

Wstaw zrzut ekranu kodu szablonu EJS listy:

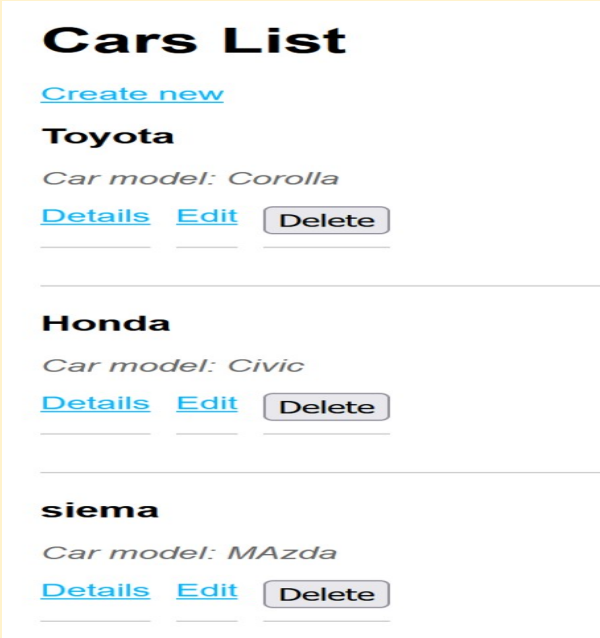
```
<div class="form-group">
  <label for="subject">Subject</label>
  <input type="text" id="subject" name="subject" value="%= car ? car.subject : '
</div>

<div class="form-group">
  <label for="content">Content</label>
  <textarea id="content" name="content">%= car ? car.content : '' %</textarea>
</div>

<div class="form-group">
  <label for="carModel">Car model</label>
  <input type="text" id="carModel" name="carModel" value="%= car ? car.carModel
</div>

<div class="form-group">
  <label></label>
  <input type="submit" value="Submit">
</div>
```

Wstaw zrzut ekranu listy (z danymi) w przeglądarce:



Punkty:	0	1
---------	---	---

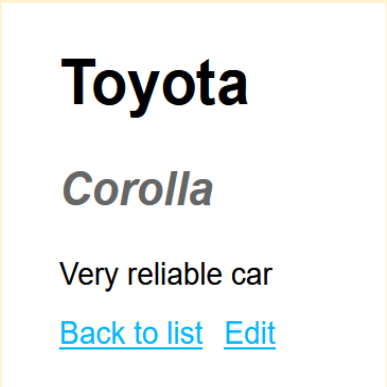
Wstaw zrzut ekranu kodu routingu i handlera podglądu pojedynczego modelu:

```
/* GET show car details. */
router.get( path:('/:id'), handlers: function(req : Request<P, ResBody, ReqBody, ReqQuery, L
    const car : Car | null = Car.find(req.params.id);
    if (!car) {
        return res.status( code: 404 ).render( view: 'error', options: { message: 'Car not found' });
    }
    res.render( view: 'cars/show', options: { title: car.subject, car: car });
});
```

Wstaw zrzut ekranu kodu szablonu EJS podglądu:

```
<%= car.subject %>
<html>
  <head>
    <title><%= car.subject %></title>
    <link rel='stylesheet' href='/stylesheets/style.css' />
  </head>
  <body>
    <h1><%= car.subject %></h1>
    <h2 class="car-model"><%= car.carModel %></h2>
    <article>
      <%= car.content %>
    </article>
    <ul class="action-list">
      <li><a href="/cars">Back to list</a></li>
      <li><a href="/cars/<%= car.id %>/edit">Edit</a></li>
    </ul>
  </body>
</html>
```

Wstaw zrzut ekranu podglądu w przeglądarce:



Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu routingu i handlera tworzenia modelu:

```
/* GET create car form. */
router.get( path: '/create', handlers: function(req : Request<P, ResBody, ReqB
    res.render( view: 'cars/create', options: { title: 'Create New Car', c
});
```

Wstaw zrzut ekranu kodu szablonu EJS tworzenia:

```
<!DOCTYPE html>
<html>
  <head>
    <title><%= title %></title>
    <link rel='stylesheet' href='/stylesheets/style.css' />
  </head>
  <body>
    <h1><%= title %></h1>
    <form action="/cars/create" method="POST">
      <%- include('_form', { car: null }) %>
    </form>
    <a href="/cars">Back to list</a>
  </body>
</html>
```

Wstaw zrzut ekranu tworzenia w przeglądarce:

Create New Car

Subject

Content

Car model

Submit

[Back to list](#)

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu routingu i handlera(ów) edycji modelu:

```
/* GET edit car form. */
router.get('/:id/edit', handlers: function(req : Request<P, ResBody>): Response {
  const car : Car | null = Car.find(req.params.id);
  if (!car) {
    return res.status(404).render('error', options: { message: 'Car not found' });
  }
  res.render('cars/edit', options: { title: 'Edit Car', car: car });
});
```

Wstaw zrzut ekranu kodu szablonu EJS edycji:

```
<!DOCTYPE html>
<html>
  <head>
    <title><%= title %></title>
    <link rel='stylesheet' href='/stylesheets/style.css' />
  </head>
  <body>
    <h1><%= title %></h1>
    <form action="/cars/<%= car.id %>/edit" method="POST">
      <%= include('_form', { car: car }) %>
    </form>
    <a href="/cars">Back to list</a>
  </body>
</html>
```

Wstaw zrzut ekranu edycji:

Edit Car

Subject

Honda

Content

Great performance

Car model

Civic

Submit

[Back to list](#)

Punkty:	0	1
---------	---	---

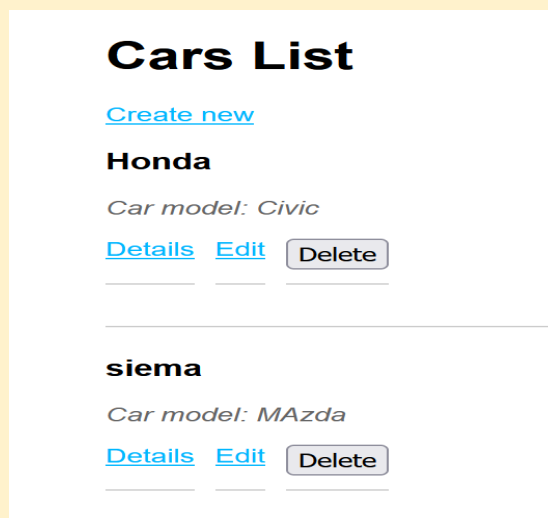
Wstaw zrzut ekranu kodu routingu i handlera kasowania:

```

/* POST delete car. */
router.post(path:('/:id/delete', handlers: function(req : Request<P, ResBody, ReqBody, ReqQue
  Car.delete(req.params.id);
  res.redirect(url: '/cars');
});

```

Wstaw zrzut ekranu działającego kasowania:



Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do swojego repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało.

Podaj link do swojego repozytorium:

<https://github.com/barczykjakub553-ui/barczykjakub553-ui.github.io/tree/main2/lab-i>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. **Nie używaj LLM.** Który z trzech poznanych dotychczas sposobów tworzenia aplikacji najbardziej przypadł Ci do gustu?

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.